

```
=====
FILE: src/app/techgen/page.tsx
=====
```

```
0001: "use client";
0002:
0003: import { MetricsTechgen } from "@components/techgen/MetricsTechgen";
0004: import React, { useMemo } from "react";
0005: import StatusUpdatesPanel from "@components/techgen/StatusUpdatesPanel";
0006: import { useGetUserApplicationIds } from "@hooks/applications/useGetUserApplications";
0007: import Link from "next/link";
0008: import { PlusIcon } from "lucide-react";
0009:
0010: export default function TechgenDashboard() {
0011:   const { userApplicationIds } = useGetUserApplicationIds();
0012:
0013:   // children gets stable reference
0014:   const applicationIds = useMemo(() => {
0015:     return userApplicationIds
0016:       ?.map((app) => app.application_id)
0017:       .filter((id): id is string => id != null && id.trim() !== "");
0018:   }, [userApplicationIds]);
0019:
0020:   return (
0021:     <div className="space-y-6">
0022:       <div className="flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-between">
0023:         <div>
0024:           <h1 className="text-2xl font-semibold text-gray-800">Dashboard</h1>
0025:           <p className="mt-1 text-sm text-gray-500">
0026:             Track your applications and recent status updates.
0027:           </p>
0028:         </div>
0029:         <Link
0030:           href="/techgen/new-application"
0031:           className="bg-brand-500 hover:bg-brand-600 inline-flex h-11 items-center justify-center gap-2 rounded-
0032: | lg px-4 text-sm font-medium text-white"
0033:         >
0034:           <PlusIcon size={18} />
0035:           Add New Application
0036:         </Link>
0037:       </div>
0038:
0039:       <div className="grid grid-cols-12 gap-4 md:gap-8">
0040:         <div className="col-span-12 xl:col-span-8">
0041:           <MetricsTechgen />
0042:         </div>
0043:
0044:         <div className="col-span-12 xl:col-span-4">
0045:           <StatusUpdatesPanel applicationIds={applicationIds ?? []} />
0046:         </div>
0047:       </div>
0048:     </div>
0049:   );
0050: }
```

```
=====
FILE: src/components/techgen/MetricsTechgen.tsx
=====
```

```
0001: "use client";
0002:
0003: import React, { useMemo } from "react";
0004: import {
0005:   FileText,
0006:   Send,
0007:   Clock3,
0008:   CheckCircle2,
0009:   Undo2,
0010:   ArrowDown,
0011:   Shield,
0012:   Shapes,
0013:   Palette,
```

```
0014:   BadgeCheck,
0015:   Copyright,
0016:   Inbox,
0017:   CalendarRange,
0018:   Layers3,
0019: } from "lucide-react";
0020: import { useGetDashboardAnalyticsTechgen } from "@/hooks/views/useGetDashboardAnalyticsTechgen";
0021: import { STATUS_ORDER } from "@/lib/dashboard/dashboard-summary";
0022: import { IpType, IP_TYPES } from "@/lib/types/ip";
0023: import { ipTypeToTitle } from "@/lib/helper/get-ip-title";
0024:
0025: type MetricCardProps = {
0026:   title: string;
0027:   value: number;
0028:   subtitle?: string;
0029:   icon: React.ElementType;
0030:   emptyText?: string;
0031:   isEmpty?: boolean;
0032: };
0033:
0034: function MetricCard({
0035:   title,
0036:   value,
0037:   subtitle,
0038:   icon: Icon,
0039:   emptyText = "No data yet",
0040:   isEmpty = false,
0041: }: MetricCardProps) {
0042:   return (
0043:     <div className="rounded-2xl border border-gray-200 bg-white p-5">
0044:       <div className="flex h-12 w-12 items-center justify-center rounded-xl bg-gray-100">
0045:         <Icon className="h-6 w-6 text-gray-600" />
0046:       </div>
0047:       <div className="mt-4">
0048:         <p className="text-sm text-gray-500">{title}</p>
0049:         <h3 className="mt-2 text-2xl font-bold text-gray-900">{value}</h3>
0050:         <p className="mt-1 text-xs text-gray-400">
0051:           {isEmpty ? emptyText : subtitle}
0052:         </p>
0053:       </div>
0054:     </div>
0055:   );
0056: };
0057: }
0058:
0059: type MetricCardSkeletonProps = {
0060:   icon: React.ElementType;
0061: };
0062:
0063: function MetricCardSkeleton({ icon }: MetricCardSkeletonProps) {
0064:   const Icon = icon;
0065:
0066:   return (
0067:     <div className="rounded-2xl border border-gray-200 bg-white p-5">
0068:       <div className="flex h-12 w-12 animate-pulse items-center justify-center rounded-xl bg-gray-100">
0069:         <Icon className="h-6 w-6 text-gray-400" />
0070:       </div>
0071:       <div className="mt-4 space-y-3">
0072:         <div className="h-4 w-24 animate-pulse rounded bg-gray-100" />
0073:         <div className="h-8 w-16 animate-pulse rounded bg-gray-200" />
0074:         <div className="h-3 w-36 animate-pulse rounded bg-gray-100" />
0075:       </div>
0076:     </div>
0077:   );
0078: };
0079: }
0080:
0081: type BreakdownItemProps = {
0082:   label: string;
0083:   value: number;
0084:   total: number;
0085:   icon?: React.ElementType;
```

```
0086: };
0087:
0088: function BreakdownItem({
0089:   label,
0090:   value,
0091:   total,
0092:   icon: Icon,
0093: }: BreakdownItemProps) {
0094:   const percentage = total > 0 ? Math.round((value / total) * 100) : 0;
0095:
0096:   return (
0097:     <div className="space-y-2 rounded-xl bg-gray-50 p-4">
0098:       <div className="flex items-center justify-between gap-3">
0099:         <div className="flex items-center gap-2 text-sm font-medium text-gray-700">
0100:           {Icon ? <Icon className="h-4 w-4 text-gray-500" /> : null}
0101:           <span>{label}</span>
0102:         </div>
0103:         <span className="text-sm font-semibold text-gray-900">{value}</span>
0104:       </div>
0105:
0106:       <div className="h-2 w-full overflow-hidden rounded-full bg-gray-200">
0107:         <div
0108:           className="bg-brand-500 h-full rounded-full"
0109:           style={{ width: `${percentage}%` }}
0110:         />
0111:       </div>
0112:
0113:       <p className="text-xs text-gray-500">
0114:         {percentage}% of total applications
0115:       </p>
0116:     </div>
0117:   );
0118: }
0119:
0120: function BreakdownItemSkeleton() {
0121:   return (
0122:     <div className="space-y-2 rounded-xl bg-gray-50 p-4">
0123:       <div className="flex items-center justify-between gap-3">
0124:         <div className="flex items-center gap-2">
0125:           <div className="h-4 w-4 animate-pulse rounded bg-gray-200" />
0126:           <div className="h-4 w-28 animate-pulse rounded bg-gray-100" />
0127:         </div>
0128:         <div className="h-4 w-8 animate-pulse rounded bg-gray-100" />
0129:       </div>
0130:
0131:       <div className="h-2 w-full overflow-hidden rounded-full bg-gray-200">
0132:         <div className="bg-brand-100 h-full w-2/3 animate-pulse rounded-full" />
0133:       </div>
0134:
0135:       <div className="h-3 w-24 animate-pulse rounded bg-gray-100" />
0136:     </div>
0137:   );
0138: }
0139:
0140: type YearItemProps = {
0141:   year: number;
0142:   total: number;
0143:   max: number;
0144: };
0145:
0146: function YearItem({ year, total, max }: YearItemProps) {
0147:   const percentage = max > 0 ? Math.round((total / max) * 100) : 0;
0148:
0149:   return (
0150:     <div className="space-y-2">
0151:       <div className="flex items-center justify-between text-sm">
0152:         <span className="font-medium text-gray-700">{year}</span>
0153:         <span className="font-semibold text-gray-900">{total}</span>
0154:       </div>
0155:
0156:       <div className="h-2 w-full overflow-hidden rounded-full bg-gray-200">
0157:         <div
```

```
0158:         className="bg-brand-500 h-full rounded-full"
0159:         style={{ width: `${percentage}%` }}
0160:       />
0161:     </div>
0162:   </div>
0163: );
0164: }
0165:
0166: function YearItemSkeleton() {
0167:   return (
0168:     <div className="space-y-2">
0169:       <div className="flex items-center justify-between">
0170:         <div className="h-4 w-14 animate-pulse rounded bg-gray-100" />
0171:         <div className="h-4 w-8 animate-pulse rounded bg-gray-100" />
0172:       </div>
0173:       <div className="h-2 w-full overflow-hidden rounded-full bg-gray-200">
0174:         <div className="bg-brand-100 h-full w-1/2 animate-pulse rounded-full" />
0175:       </div>
0176:     </div>
0177:   );
0178: };
0179: }
0180:
0181: type EmptyPanelProps = {
0182:   title: string;
0183:   description: string;
0184:   icon: React.ElementType;
0185: };
0186:
0187: function EmptyPanel({ title, description, icon: Icon }: EmptyPanelProps) {
0188:   return (
0189:     <div className="flex min-h-[260px] flex-col items-center justify-center rounded-2xl border border-dashed
0190: | border-gray-200 bg-white px-6 py-10 text-center">
0191:       <div className="flex h-12 w-12 items-center justify-center rounded-xl bg-gray-100">
0192:         <Icon className="h-6 w-6 text-gray-500" />
0193:       </div>
0194:       <h3 className="mt-4 text-base font-semibold text-gray-900">{title}</h3>
0195:       <p className="mt-2 max-w-sm text-sm text-gray-500">{description}</p>
0196:     </div>
0197:   );
0198: }
0199:
0200: function BreakdownPanelSkeleton({
0201:   title,
0202:   description,
0203:   icon,
0204:   children,
0205: }): {
0206:   title: string;
0207:   description: string;
0208:   icon: React.ElementType;
0209:   children: React.ReactNode;
0210: }) {
0211:   const Icon = icon;
0212:
0213:   return (
0214:     <div className="rounded-2xl border border-gray-200 bg-white p-5">
0215:       <div className="mb-4">
0216:         <div className="flex items-center gap-2">
0217:           <div className="bg-brand-50 flex h-8 w-8 animate-pulse items-center justify-center rounded-lg">
0218:             <Icon className="text-brand-600 h-4 w-4" />
0219:           </div>
0220:           <div className="h-5 w-40 animate-pulse rounded bg-gray-100" />
0221:         </div>
0222:         <div className="mt-2 h-4 w-64 animate-pulse rounded bg-gray-100" />
0223:       </div>
0224:       <div className="space-y-3">{children}</div>
0225:     </div>
0226:   );
0227: }
0228:
```

```
0229: function DashboardLoadingState() {
0230:   return (
0231:     <div className="space-y-6">
0232:       <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 xl:grid-cols-3">
0233:         <MetricCardSkeleton icon={FileText} />
0234:         <MetricCardSkeleton icon={Send} />
0235:         <MetricCardSkeleton icon={Clock3} />
0236:         <MetricCardSkeleton icon={CheckCircle2} />
0237:         <MetricCardSkeleton icon={Undo2} />
0238:         <MetricCardSkeleton icon={ArrowDown} />
0239:       </div>
0240:
0241:       <div className="grid grid-cols-1 gap-4 xl:grid-cols-2">
0242:         <BreakdownPanelSkeleton
0243:           title="Applications by IP Type"
0244:           description="Loading filing distribution across IP categories"
0245:           icon={Layers3}
0246:         >
0247:           <BreakdownItemSkeleton />
0248:           <BreakdownItemSkeleton />
0249:           <BreakdownItemSkeleton />
0250:           <BreakdownItemSkeleton />
0251:           <BreakdownItemSkeleton />
0252:         </BreakdownPanelSkeleton>
0253:
0254:         <BreakdownPanelSkeleton
0255:           title="Applications by Year"
0256:           description="Loading yearly filing activity"
0257:           icon={CalendarRange}
0258:         >
0259:           <YearItemSkeleton />
0260:           <YearItemSkeleton />
0261:           <YearItemSkeleton />
0262:           <YearItemSkeleton />
0263:         </BreakdownPanelSkeleton>
0264:       </div>
0265:     </div>
0266:   );
0267: }
0268:
0269: export function MetricsTechgen() {
0270:   const { data, isLoading } = useGetDashboardAnalyticsTechgen();
0271:
0272:   const summary = useMemo(() => {
0273:     const temp = {
0274:       totalApplications: 0,
0275:       filed: 0,
0276:       pending: 0,
0277:       granted: 0,
0278:       withdrawn: 0,
0279:       downgraded: 0,
0280:     };
0281:
0282:     if (!data || data.length === 0) {
0283:       return temp;
0284:     }
0285:
0286:     temp.totalApplications = data.reduce((acc, row) => {
0287:       return acc + (row.total ?? 0);
0288:     }, 0);
0289:
0290:     for (const status of STATUS_ORDER) {
0291:       const filteredRows = data.filter(
0292:         (row) => row.dashboard_status === status,
0293:       );
0294:
0295:       temp[status as keyof typeof temp] = filteredRows.reduce((acc, row) => {
0296:         return acc + (row.total ?? 0);
0297:       }, 0);
0298:     }
0299:
0300:     return temp;
```

```
0301:   }, [data]);
0302:
0303:   const ipTypeIcon = {
0304:     patent: Shield,
0305:     utility_model: Shapes,
0306:     industrial_design: Palette,
0307:     copyright: Copyright,
0308:     trademark: BadgeCheck,
0309:   };
0310:
0311:   const applicationsByType = useMemo(() => {
0312:     const temp: { label: string; value: number; icon: React.ElementType }[] =
0313:       [];
0314:
0315:     if (!data || data.length === 0) {
0316:       return temp;
0317:     }
0318:
0319:     for (const ipType of IP_TYPES) {
0320:       const filteredRows = data.filter((row) => row.ip_type === ipType);
0321:
0322:       const value = filteredRows.reduce((acc, row) => {
0323:         return acc + (row.total ?? 0);
0324:       }, 0);
0325:
0326:       temp.push({
0327:         label: ipTypeToTitle(ipType),
0328:         value,
0329:         icon: ipTypeIcon[ipType],
0330:       });
0331:     }
0332:
0333:     return temp;
0334:   }, [data]);
0335:
0336:   const applicationsByYear = useMemo(() => {
0337:     const temp: Record<number, number> = {};
0338:
0339:     if (!data || data.length === 0) {
0340:       return temp;
0341:     }
0342:
0343:     data.forEach((item) => {
0344:       if (!item || !item.year) return;
0345:
0346:       temp[item.year] = (temp[item.year] ?? 0) + (item.total ?? 0);
0347:     });
0348:
0349:     return temp;
0350:   }, [data]);
0351:
0352:   if (isLoading) {
0353:     return <DashboardLoadingState />;
0354:   }
0355:
0356:   const hasApplications = summary.totalApplications > 0;
0357:   const highestYearTotal =
0358:     Object.keys(applicationsByYear).length > 0
0359:       ? Math.max(...Object.values(applicationsByYear))
0360:       : 0;
0361:
0362:   return (
0363:     <div className="space-y-6">
0364:       <div className="grid grid-cols-2 gap-4 sm:grid-cols-3">
0365:         <MetricCard
0366:           title="Total Applications"
0367:           value={summary.totalApplications}
0368:           subtitle="All IP applications recorded"
0369:           emptyText="No applications recorded yet"
0370:           icon={FileText}
0371:           isEmpty={!hasApplications}
0372:         />
```

```
0373:         <MetricCard
0374:             title="Filed"
0375:             value={summary.filed}
0376:             subtitle="Applications formally filed"
0377:             emptyText="No filed applications yet"
0378:             icon={Send}
0379:             isEmpty={!hasApplications}
0380:         />
0381:         <MetricCard
0382:             title="Pending"
0383:             value={summary.pending}
0384:             subtitle="Currently under review or processing"
0385:             emptyText="No pending applications yet"
0386:             icon={Clock3}
0387:             isEmpty={!hasApplications}
0388:         />
0389:         <MetricCard
0390:             title="Granted"
0391:             value={summary.granted}
0392:             subtitle="Applications successfully granted"
0393:             emptyText="No granted applications yet"
0394:             icon={CheckCircle2}
0395:             isEmpty={!hasApplications}
0396:         />
0397:         <MetricCard
0398:             title="Withdrawn"
0399:             value={summary.withdrawn}
0400:             subtitle="Applications withdrawn"
0401:             emptyText="No withdrawn applications yet"
0402:             icon={Undo2}
0403:             isEmpty={!hasApplications}
0404:         />
0405:         <MetricCard
0406:             title="Downgraded"
0407:             value={summary.downgraded}
0408:             subtitle="Applications moved to a lower IP type"
0409:             emptyText="No downgraded applications yet"
0410:             icon={ArrowDown}
0411:             isEmpty={!hasApplications}
0412:         />
0413:     </div>
0414:
0415:     <div className="grid grid-cols-1 gap-4 xl:grid-cols-2">
0416:         <div className="rounded-2xl border border-gray-200 bg-white p-5">
0417:             <div className="mb-4">
0418:                 <div className="flex items-center gap-2">
0419:                     <div className="bg-brand-50 flex h-8 w-8 items-center justify-center rounded-lg">
0420:                         <Layers3 className="text-brand-600 h-4 w-4" />
0421:                     </div>
0422:                     <h2 className="text-lg font-semibold text-gray-900">
0423:                         Applications by IP Type
0424:                     </h2>
0425:                 </div>
0426:                 <p className="mt-2 text-sm text-gray-500">
0427:                     Distribution of your filings across IP categories
0428:                 </p>
0429:             </div>
0430:
0431:             {hasApplications && applicationsByType.length > 0 ? (
0432:                 <div className="space-y-3">
0433:                     {applicationsByType.map((item) => (
0434:                         <BreakdownItem
0435:                             key={item.label}
0436:                             label={item.label}
0437:                             value={item.value}
0438:                             total={summary.totalApplications}
0439:                             icon={item.icon}
0440:                         />
0441:                     ))}
0442:                 </div>
0443:             ) : (
0444:                 <EmptyPanel
```

```

0445:         title="No IP type data yet"
0446:         description="Once applications are recorded, their distribution across patent, utility model,
| industrial design, trademark, and copyright will appear here."
0447:         icon={Layers3}
0448:     />
0449:     )}
0450: </div>
0451:
0452: <div className="rounded-2xl border border-gray-200 bg-white p-5">
0453:     <div className="mb-4">
0454:         <div className="flex items-center gap-2">
0455:             <div className="bg-brand-50 flex h-8 w-8 items-center justify-center rounded-lg">
0456:                 <CalendarRange className="text-brand-600 h-4 w-4" />
0457:             </div>
0458:             <h2 className="text-lg font-semibold text-gray-900">
0459:                 Applications by Year
0460:             </h2>
0461:         </div>
0462:         <p className="mt-2 text-sm text-gray-500">
0463:             Filing activity over the years
0464:         </p>
0465:     </div>
0466:
0467:     {hasApplications && Object.keys(applicationsByYear).length > 0 ? (
0468:         <div className="space-y-4">
0469:             {Object.entries(applicationsByYear).map(([year, total]) => (
0470:                 <YearItem
0471:                     key={year}
0472:                     year={Number(year)}
0473:                     total={total}
0474:                     max={highestYearTotal}
0475:                 />
0476:             )}
0477:         </div>
0478:     ) : (
0479:         <EmptyPanel
0480:             title="No yearly activity yet"
0481:             description="Application activity by year will appear here once at least one IP application has
| been recorded."
0482:             icon={CalendarRange}
0483:         />
0484:     )}
0485: </div>
0486: </div>
0487:
0488: {!hasApplications && (
0489:     <div className="rounded-2xl border border-dashed border-gray-200 bg-white p-6">
0490:         <div className="flex items-start gap-4">
0491:             <div className="flex h-12 w-12 items-center justify-center rounded-xl bg-gray-100">
0492:                 <Inbox className="h-6 w-6 text-gray-500" />
0493:             </div>
0494:             <div>
0495:                 <h2 className="text-lg font-semibold text-gray-900">
0496:                     No applications yet
0497:                 </h2>
0498:                 <p className="mt-1 text-sm text-gray-500">
0499:                     When this TechGen starts submitting intellectual property
0500:                     applications, the dashboard metrics and breakdowns will appear
0501:                     here.
0502:                 </p>
0503:             </div>
0504:         </div>
0505:     </div>
0506: )}
0507: </div>
0508: );
0509: }

```

```

=====
FILE: src/components/techgen/StatusUpdatesPanel.tsx
=====

```

```

0001: "use client";

```

```
0002:
0003: import Link from "next/link";
0004: import { ReactNode } from "react";
0005:
0006: import { StatusType } from "@lib/types/ip";
0007: import { STATUS_LABELS } from "@lib/helper/status-labels";
0008: import { formatDate, formatDateTime } from "@lib/helper/format-date";
0009: import { useGetMultipleApplicationStatuses } from "@hooks/status/useGetMultipleApplicationStatuses";
0010: import { useGetMultipleAppById } from "@hooks/applications/useGetMultipleApplicationById";
0011: import { IprStatusType } from "@lib/types/status";
0012: import { Clock3 } from "lucide-react";
0013:
0014: interface PropsInterface {
0015:   applicationIds: string[];
0016: }
0017:
0018: type StatusListItem = {
0019:   applicationId: string;
0020:   projectTitle: string;
0021:   status: IprStatusType["Row"];
0022:   timestamp: number;
0023: };
0024:
0025: export default function StatusUpdatesPanel(props: PropsInterface) {
0026:   const { applicationIds } = props;
0027:
0028:   const { statuses, isLoading: isStatusesLoading } =
0029:     useGetMultipleApplicationStatuses({
0030:       applicationIds,
0031:       isLatest: true,
0032:     });
0033:
0034:   const { applications } = useGetMultipleAppById({
0035:     applicationIds,
0036:   });
0037:
0038:   if (isStatusesLoading.length > 0 && isStatusesLoading.every(Boolean)) {
0039:     return <StatusUpdateContainer>Fetching statuses...</StatusUpdateContainer>;
0040:   }
0041:
0042:   if (
0043:     !statuses ||
0044:     statuses.length < 1 ||
0045:     !applications ||
0046:     applications.length < 1
0047:   ) {
0048:     return (
0049:       <StatusUpdateContainer>
0050:         <EmptyStateText>No status history available.</EmptyStateText>
0051:       </StatusUpdateContainer>
0052:     );
0053:   }
0054:
0055:   const statusList: StatusListItem[] = statuses
0056:     .map((status, i) => {
0057:       if (Array.isArray(status)) return null;
0058:
0059:       const application = applications[i];
0060:       if (!status || !application || !status.created_at) return null;
0061:
0062:       const timestamp = Date.parse(status.created_at);
0063:       if (isNaN(timestamp)) return null;
0064:
0065:       return {
0066:         applicationId: application.id,
0067:         projectTitle: application.project_title,
0068:         status,
0069:         timestamp,
0070:       };
0071:     })
0072:     .filter((item): item is StatusListItem => item !== null)
0073:     .sort((a, b) => b.timestamp - a.timestamp);
```

```
0074:
0075:   return (
0076:     <StatusUpdateContainer>
0077:       <div className="mt-3 max-h-[24rem] overflow-y-auto pr-1">
0078:         <div className="relative pl-5">
0079:           <div className="absolute top-1 bottom-1 left-[7px] w-px bg-[var(--color-gray-200)]" />
0080:
0081:           <div className="space-y-3">
0082:             {statusList.map((item) => {
0083:               const label =
0084:                 STATUS_LABELS[item.status.status_type as StatusType] ??
0085:                 item.status.status_type;
0086:
0087:               return (
0088:                 <TimelineItem
0089:                   key={`_${item.applicationId}-${item.status.created_at}`}
0090:                   applicationId={item.applicationId}
0091:                   projectTitle={item.projectTitle}
0092:                   label={label}
0093:                   createdAt={item.status.created_at!}
0094:                   note={item.status.note}
0095:                   deadline={item.status.deadline}
0096:                 />
0097:               );
0098:             })}
0099:           </div>
0100:         </div>
0101:       </div>
0102:     </StatusUpdateContainer>
0103:   );
0104: }
0105:
0106: interface TimelineItemProps {
0107:   applicationId: string;
0108:   projectTitle: string;
0109:   label: string;
0110:   createdAt: string;
0111:   note?: string | null;
0112:   deadline?: string | null;
0113: }
0114:
0115: function TimelineItem({
0116:   applicationId,
0117:   projectTitle,
0118:   label,
0119:   createdAt,
0120:   note,
0121:   deadline,
0122: }: TimelineItemProps) {
0123:   return (
0124:     <div className="relative">
0125:       <div className="absolute top-2.5 -left-5 h-3.5 w-3.5 rounded-full border border-[var(--color-gray-300)]
0126: | bg-white">
0127:         <div className="absolute top-1/2 left-1/2 h-1.5 w-1.5 -translate-x-1/2 -translate-y-1/2 rounded-full
0128: | bg-[var(--color-gray-400)]" />
0129:       </div>
0130:       <div className="rounded-xl border border-[var(--color-gray-200)] bg-white px-3 py-2.5
0131: | hover:border-[var(--color-gray-300)]">
0132:         <div className="flex items-start justify-between gap-3">
0133:           <div className="min-w-0 flex-1">
0134:             <div className="flex items-start justify-between gap-3">
0135:               <Link
0136:                 href={`/${applications}/${applicationId}`}
0137:                 className="truncate text-sm font-medium text-[var(--color-gray-900)] hover:text-[var(--color-
0138: | brand-600)]"
0139:               >
0140:                 {projectTitle}
0141:               </Link>
0142:             <p className="shrink-0 text-[11px] text-[var(--color-gray-500)]">
0143:               {formatDateTime(createdAt)}
```

```
0142:         </p>
0143:     </div>
0144:
0145:     <div className="mt-1 flex items-center gap-2">
0146:         <p className="text-xs font-medium text-[var(--color-gray-700)]">
0147:             {label}
0148:         </p>
0149:
0150:         <span className="h-1 w-1 rounded-full bg-[var(--color-gray-300)]" />
0151:
0152:         <Link
0153:             href={'/techgen/view-application?applicationID=${applicationId}'}
0154:             className="text-xs text-[var(--color-brand-600)] hover:text-[var(--color-brand-700)]"
0155:         >
0156:             View
0157:         </Link>
0158:     </div>
0159:
0160:     {note && (
0161:         <p className="mt-2 line-clamp-2 text-xs text-[var(--color-gray-600)]">
0162:             {note}
0163:         </p>
0164:     )}
0165:
0166:     {deadline && (
0167:         <p className="mt-1 text-xs text-[var(--color-error-600)]">
0168:             Deadline: {formatDate(deadline)}
0169:         </p>
0170:     )}
0171: </div>
0172: </div>
0173: </div>
0174: </div>
0175: );
0176: }
0177:
0178: function StatusUpdateContainer({ children }: { children: ReactNode }) {
0179:     return (
0180:         <div className="rounded-2xl border border-[var(--color-gray-200)] bg-white p-4">
0181:             <div className="mb-4">
0182:                 <div className="flex items-center gap-2">
0183:                     <div className="bg-brand-50 flex h-8 w-8 items-center justify-center rounded-lg">
0184:                         <Clock3 className="text-brand-600 h-4 w-4" />
0185:                     </div>
0186:                     <h2 className="text-lg font-semibold text-gray-900">
0187:                         Recent Updates
0188:                     </h2>
0189:                 </div>
0190:                 <p className="mt-2 text-sm text-gray-500">
0191:                     Latest update for each application
0192:                 </p>
0193:             </div>
0194:             {children}
0195:         </div>
0196:     );
0197: }
0198:
0199: function EmptyStateText({ children }: { children: ReactNode }) {
0200:     return (
0201:         <div className="flex min-h-[120px] items-center justify-center rounded-xl border border-dashed
0202: | border-[var(--color-gray-200)] bg-[var(--color-gray-25)] px-4 text-sm text-[var(--color-gray-500)]">
0203:             {children}
0204:         </div>
0205:     );
0206: }
```

=====

FILE: src/hooks/applications/useGetUserApplications.ts

=====

```
0001: import { getUserApplicationIds } from "@services/application/get-user-applications";
0002: import { useQuery } from "@tanstack/react-query";
0003:
```

```
0004: export function useGetUserApplicationIds() {
0005:   const { data, isLoading } = useQuery({
0006:     queryKey: ['user-applications'],
0007:     queryFn: getUserApplicationIds
0008:   })
0009:
0010:   return {userApplicationIds: data, isLoading}
0011: }
```

```
=====
FILE: src/services/application/get-user-applications.ts
=====
```

```
0001: import { supabaseClient as supabase } from "@lib/supabase";
0002:
0003: export const getUserApplicationIds = async function() {
0004:   const { data: { user } } = await supabase.auth.getUser();
0005:   const userId = user?.id;
0006:
0007:   if (!userId) {
0008:     throw new Error("Cannot find user id");
0009:   }
0010:
0011:   const {data, error} = await supabase
0012:     .schema("private")
0013:     .from("inventors")
0014:     .select("application_id")
0015:     .eq("techgen_id", userId)
0016:
0017:   if (error) {
0018:     throw new Error(error.message);
0019:   }
0020:
0021:   return data;
0022: }
```

```
=====
FILE: src/hooks/views/useGetDashboardAnalyticsTechgen.ts
=====
```

```
0001: import { getDashboardAnalyticsTechgen } from "@services/views/get-dashboard-analytics-techgen";
0002: import { useQuery } from "@tanstack/react-query";
0003:
0004: export function useGetDashboardAnalyticsTechgen(
0005: ) {
0006:   const { data, isPending } = useQuery({
0007:     queryKey: ["dashboard-analytics-techgen"],
0008:     queryFn: () => getDashboardAnalyticsTechgen(),
0009:   });
0010:
0011:   return { data, isLoading: isPending };
0012: }
```

```
=====
FILE: src/services/views/get-dashboard-analytics-techgen.ts
=====
```

```
0001: import { supabaseClient as supabase } from "@lib/supabase";
0002:
0003: export const getDashboardAnalyticsTechgen = async function (
0004: ) {
0005:   const { data: {user}} = await supabase.auth.getUser();
0006:
0007:   if (!user) {
0008:     throw new Error("User not found.");
0009:   }
0010:
0011:   const { data, error } = await supabase
0012:     .from("v_dashboard_analytics_techgen")
0013:     .select()
0014:     .eq("techgen_id", user.id)
0015:     .order("year", { ascending: false });
0016:
0017:   if (error) {
0018:     throw new Error(error.message);
0019:   }
0020: }
```

```
0019:   }  
0020:  
0021:   return data;  
0022: };
```

```
=====
```

FILE: src/lib/dashboard/dashboard-summary.ts

```
=====
```

```
0001: import { DashboardAnalyticsType } from "@lib/types/views";  
0002: import { ipTypeToTitle } from "../helper/get-ip-title";  
0003: import { IP_TYPES } from "../types/ip";  
0004:  
0005: export const STATUS_ORDER = [  
0006:   "filed",  
0007:   "pending",  
0008:   "granted",  
0009:   "withdrawn",  
0010:   "downgraded",  
0011: ] as const;  
0012:  
0013: export type DashboardStatus = (typeof STATUS_ORDER)[number];  
0014:  
0015: export const DASHBOARD_STATUS_LABELS: Record<DashboardStatus, string> = {  
0016:   filed: "Filed",  
0017:   pending: "Pending",  
0018:   granted: "Granted",  
0019:   withdrawn: "Withdrawn",  
0020:   downgraded: "Downgraded",  
0021: };  
0022:  
0023: export type SummaryTableRow = {  
0024:   year: number;  
0025:   ipType: string;  
0026:   filed: number;  
0027:   pending: number;  
0028:   granted: number;  
0029:   withdrawn: number;  
0030:   downgraded: number;  
0031:   total: number;  
0032: };  
0033:  
0034: export type SummaryTotals = {  
0035:   filed: number;  
0036:   pending: number;  
0037:   granted: number;  
0038:   withdrawn: number;  
0039:   downgraded: number;  
0040:   total: number;  
0041: };  
0042:  
0043: export const formatIpTypeLabel = (ipType: string) => {  
0044:   return (  
0045:     ipTypeToTitle(ipType) ??  
0046:     ipType  
0047:       .split("_")  
0048:       .map((part) => part.charAt(0).toUpperCase() + part.slice(1))  
0049:       .join(" ")  
0050:   );  
0051: };  
0052:  
0053: export const buildSummaryTableRows = (  
0054:   filteredData: DashboardAnalyticsType[],  
0055: ): SummaryTableRow[] => {  
0056:   const groupedRows = new Map<string, Omit<SummaryTableRow, "total">>();  
0057:  
0058:   filteredData.forEach((item) => {  
0059:     if (!item.ip_type || item.year === null) return;  
0060:     if (!STATUS_ORDER.includes(item.dashboard_status as DashboardStatus)) {  
0061:       return;  
0062:     }  
0063:  
0064:     const year = Number(item.year);
```

```
0065:     const key = `${year}:${item.ip_type}`;
0066:
0067:     if (!groupedRows.has(key)) {
0068:       groupedRows.set(key, {
0069:         year,
0070:         ipType: item.ip_type,
0071:         filed: 0,
0072:         pending: 0,
0073:         granted: 0,
0074:         withdrawn: 0,
0075:         downgraded: 0,
0076:       });
0077:     }
0078:
0079:     const row = groupedRows.get(key)!;
0080:     const status = item.dashboard_status as DashboardStatus;
0081:     row[status] += Number(item.total ?? 0);
0082:   });
0083:
0084:   return Array.from(groupedRows.values())
0085:     .map((row) => ({
0086:       ...row,
0087:       total:
0088:         row.filed + row.pending + row.granted + row.withdrawn + row.downgraded,
0089:     }))
0090:     .sort((a, b) => {
0091:       if (a.year !== b.year) return a.year - b.year;
0092:
0093:       const aIndex = IP_TYPES.indexOf(a.ipType as (typeof IP_TYPES)[number]);
0094:       const bIndex = IP_TYPES.indexOf(b.ipType as (typeof IP_TYPES)[number]);
0095:
0096:       if (aIndex !== -1 && bIndex !== -1) return aIndex - bIndex;
0097:       if (aIndex !== -1) return -1;
0098:       if (bIndex !== -1) return 1;
0099:
0100:       return a.ipType.localeCompare(b.ipType);
0101:     });
0102: };
0103:
0104: export const buildSummaryTotals = (rows: SummaryTableRow[]): SummaryTotals => {
0105:   return rows.reduce(
0106:     (acc, row) => {
0107:       acc.filed += row.filed;
0108:       acc.pending += row.pending;
0109:       acc.granted += row.granted;
0110:       acc.withdrawn += row.withdrawn;
0111:       acc.downgraded += row.downgraded;
0112:       acc.total += row.total;
0113:       return acc;
0114:     },
0115:     {
0116:       filed: 0,
0117:       pending: 0,
0118:       granted: 0,
0119:       withdrawn: 0,
0120:       downgraded: 0,
0121:       total: 0,
0122:     },
0123:   );
0124: };
```

=====

FILE: src/hooks/status/useGetMultipleApplicationStatuses.ts

=====

```
0001: import { getApplicationStatuses } from "@services/status/get-application-statuses";
0002: import { useQueries } from "@tanstack/react-query";
0003:
0004: interface PropsInterface {
0005:   applicationIds: string[],
0006:   isLatest?: boolean;
0007: }
0008:
```

```

0009: export function useGetMultipleApplicationStatuses(props: PropsInterface) {
0010:     const {applicationIds, isLatest } = props;
0011:     const validApplicationIds = applicationIds.filter(id => id!=null && id!="");
0012:
0013:     const queries = useQueries(
0014:         {
0015:             queries: validApplicationIds.map((applicationId) => (
0016:                 {
0017:                     queryKey: ["multiple-status", applicationId, isLatest],
0018:                     queryFn: () => getApplicationStatuses({applicationId, isLatest}),
0019:                 }
0020:             ))
0021:         }
0022:     )
0023:
0024:     return { statuses: queries.map((q) => q.data), isLoading: queries.map((q) => q.isLoading)};
0025: }

```

```

=====
FILE: src/services/status/get-application-statuses.ts
=====

```

```

0001: import { supabaseClient as supabase } from "@lib/supabase";
0002:
0003: interface PropsInterface {
0004:     applicationId: string,
0005:     isLatest?: boolean,
0006: }
0007:
0008: export const getApplicationStatuses = async function(props: PropsInterface) {
0009:     const { applicationId, isLatest = false } = props;
0010:
0011:     if (!applicationId) {
0012:         throw new Error("Invalid application id.")
0013:     }
0014:
0015:     const { data, error } = await supabase
0016:         .schema("private")
0017:         .from("ipr_statuses")
0018:         .select()
0019:         .eq("application_id", applicationId)
0020:         .order("created_at", {ascending: false});
0021:
0022:     if (error) {
0023:         throw new Error(error.message);
0024:     }
0025:
0026:     if (isLatest) {
0027:         if (!data[0]) {
0028:             throw new Error("No application with that id.")
0029:         }
0030:
0031:         return data[0];
0032:     }
0033:
0034:     return data;
0035: }

```

```

=====
FILE: src/hooks/applications/useGetMultipleApplicationById.ts
=====

```

```

0001: import { useQueries, useQuery } from '@tanstack/react-query';
0002: import { getApplicationById } from '@services/application/get-application-by-id';
0003:
0004:
0005: interface UseGetApplicationByIdProp {
0006:     applicationIds: string[];
0007: }
0008:
0009: export function useGetMultipleAppById(props: UseGetApplicationByIdProp) {
0010:     const { applicationIds } = props;
0011:
0012:     const query = useQueries(

```

```
0013:      {
0014:          queries: applicationIds.map((applicationId) => (
0015:              {
0016:                  queryKey: ['multiple-application', applicationId],
0017:                  queryFn: () => getApplicationById({id: applicationId}),
0018:              }
0019:          ))
0020:      }
0021:  );
0022:
0023:  return {applications: query.map((q) => q.data), isLoading: query.map((q) => q.isLoading)};
0024: }
```

=====

FILE: src/services/application/get-application-by-id.ts

=====

```
0001: import { supabaseClient as supabase } from "@lib/supabase"
0002:
0003: interface GetApplicationByIdProps {
0004:     id: string;
0005: }
0006:
0007: export const getApplicationById = async (props: GetApplicationByIdProps) => {
0008:     const { id } = props;
0009:     const {data, error} = await supabase.schema("private").from("ipr_applications").select().eq("id",
0010: | id).single();
0011:     if (error) {
0012:         throw new Error(error.message);
0013:     }
0014:
0015:     return data;
0016: }
```

=====

FILE: src/lib/helper/get-ip-title.ts

=====

```
0001: import { IpType } from "@lib/types/ip";
0002:
0003:
0004: export const ipTypeToTitle = (ipType: IpType | null | string): string => {
0005:     switch (ipType) {
0006:         case 'utility_model':
0007:             return 'Utility Model';
0008:         case 'industrial_design':
0009:             return 'Industrial Design';
0010:         case 'trademark':
0011:             return 'Trademark';
0012:         case 'copyright':
0013:             return 'Copyright';
0014:         case 'patent':
0015:             return 'Patent'
0016:         default:
0017:             return '';
0018:     }
0019: }
```

=====

FILE: src/lib/helper/status-labels.ts

=====

```
0001:
0002: import { StatusType } from "../types/ip";
0003:
0004: export const STATUS_LABELS: Record<StatusType, string> = {
0005:     draft_classification: 'Draft Classification',
0006:     draft_idf: 'Draft IDF',
0007:     submitted_to_ttbd: 'Submitted to TTBD',
0008:     under_ttbd_review: 'Under TTBD Review',
0009:     prior_art_search: 'Prior Art Search',
0010:     draft_application: 'Draft Application',
0011:     filed_with_ipophil: 'Filed with IPOPHIL',
0012:     wait_notice_publication: 'Waiting for Notice of Publication',
```

```
0013: wait_substantive_exam_report: 'Waiting for Substantive Exam Report',
0014: resolve_ser_defects: 'Resolve SER Defects',
0015: downgraded_to_um: 'Downgraded to UM',
0016: wait_notice_of_issuance: 'Waiting for Notice of Issuance',
0017: req_cert_of_registration: 'Request Certificate of Registration',
0018: wait_cert_of_registration: 'Waiting for Certificate of Registration',
0019: registered: 'Registered',
0020: published: 'Published',
0021: wait_formality_exam_report: 'Waiting for Formality Exam Report',
0022: resolve_fer_defects: 'Resolve FER Defects',
0023: request_revival: 'Request Revival',
0024: '2nd_publication': '2nd Publication',
0025: prepare_nice_classification: 'Prepare Nice Classification',
0026: approve_nice_classification: 'Approve Nice Classification',
0027: wait_registrability_report: 'Waiting for Registrability Report',
0028: resolve_rr_defects: 'Resolve RR Defects',
0029: techgen_sign: 'TechGen Sign',
0030: chancellor_sign: 'Chancellor Sign the Deed of Assignment',
0031: notarization: 'Notarization',
0032: wait_notice_of_action: 'Waiting for Notice of Action',
0033: discussing_downgrade: 'Discussing Downgrade',
0034: resolve_additional_requirements: 'Resolve Additional Requirements',
0035: wait_statement_of_acc: 'Waiting for Statement of Account from IPOPHL',
0036: pay_fee_application: 'Pay Application Fee',
0037: mailed_to_ipophl: 'Mailing original Signed Copyright Documents to IPOPHL',
0038: closed: 'Closed',
0039: req_for_issuance_of_cert_and_2nd_publication: 'Request for Issuance of Certificate and 2nd Publication',
0040: endorse_copyright: 'Endorse Copyright Application to the Office of the Chancellor',
0041: print_copyright_forms: 'Print Copyright Forms',
0042: submit_to_ovcre: 'Submit to OVCRE for Endorsement',
0043: request_substantive_exam_report: 'Request Substantive Exam Report',
0044: removal_from_record: 'Removal from Record',
0045: expired: 'Expired',
0046: };
```

=====

FILE: src/lib/helper/format-date.ts

=====

```
0001: import { differenceInCalendarDays, isToday, format } from "date-fns";
0002:
0003: export const toSupabaseDate = (date: Date) => format(date, "yyyy-MM-dd");
0004:
0005: export const formatDate = (date: string | Date) =>
0006:   new Date(date).toLocaleDateString(undefined, {
0007:     month: "short",
0008:     day: "numeric",
0009:     year: "numeric",
0010:   });
0011:
0012: export const fromSupabaseDate = (s: string) => {
0013:   const [y, m, d] = s.split("-").map(Number);
0014:   return new Date(y, m - 1, d); // local date, no timezone shift
0015: };
0016:
0017: export const toSupabaseTimestamp = (d: Date) => d.toISOString();
0018: export const fromSupabaseTimestamp = (s: string) => new Date(s);
0019:
0020: export const formatTime = (date: string | Date) =>
0021:   new Date(date).toLocaleTimeString(undefined, {
0022:     hour: "2-digit",
0023:     minute: "2-digit"
0024:   });
0025:
0026: export const formatDateTime = (date: string | Date) =>
0027:   new Date(date).toLocaleString(undefined, {
0028:     month: "short",
0029:     day: "numeric",
0030:     year: "numeric",
0031:     hour: "2-digit",
0032:     minute: "2-digit",
0033:   });
0034:
```

```
0035: export const daysDelayed = (targetISO: string) => {
0036:   const diff = differenceInCalendarDays(new Date(), new Date(targetISO));
0037:   return Math.max(diff, 0);
0038: }
0039:
0040: export const formatSmartDate = (iso: string) => {
0041:   const d = new Date(iso);
0042:   return isToday(d) ? 'Today at ${formatTime(iso)}' : formatDate(iso);
0043: }
```